SkillFlow

Product Requirements Document

Version 1.0 | June 2026

Field	Value	Field	Value
Status	Draft	Author	SkillFlow Team
Target Users	AI agent developers	Platform	CLI + Web (Phase 5)
Core Problem	Skill overload and pipeline chaos	Key Metric	Time-to-right-skill < 5s

1. Why We Are Building This

"A developer with 100 skills and no map spends more time finding tools than using them."

The AI agent ecosystem has an infrastructure problem. The community has built hundreds of specialized skills (composable instruction sets that teach AI agents how to perform specific tasks), but there is no operating system for managing them. Developers accumulate skills the way early web developers accumulated npm packages: eagerly, without structure, and with escalating consequences. The result is skill sprawl: a growing directory of unclassified, undocumented, and incompatible skills that slow down every workflow instead of accelerating it.

The problem has three distinct dimensions that compound each other. **The discovery problem** is the most immediate: when you have 100+ skills installed, finding the right one for a given task requires scanning the entire list or relying on memory. Research on Hick's Law shows that decision time increases logarithmically with the number of choices, and the Iyengar/Lepper jam study demonstrated that 24 options yield a 3% selection rate compared to 30% for 6 options. The math is unforgiving: unorganized skills produce organized failure. **The composition problem** is deeper: even after finding a skill, developers have no way to know which other skills can be chained before or after it, because skills do not declare their input/output contracts or their composability traits. **The activation problem** is the silent killer: loading all skills into an agent's context window pollutes the agent's reasoning, causing it to suggest irrelevant skills or hallucinate capabilities that do not exist. The agent becomes

less competent as you add more skills, which is the opposite of the intended effect.

These three problems are not theoretical. In interviews with active skill users, the most common complaint is not "I don't have enough skills" but "I can't find the skill I know I have." The second most common complaint is "I don't know if these two skills work together." The third is "My agent keeps trying to use the wrong skill for the job." These are systemic problems that require a systemic solution, not more skills or better search, but a fundamentally different way of organizing, visualizing, and composing skills.

1.1 The Three Failures of the Status Quo

Failure Mode	Symptom	Root Cause	Impact
Discovery Failure	Developer scans 100+ skill names to find one, often failing	No taxonomy, no visual cues, flat alphabetical list	15-30 seconds wasted per skill lookup; wrong skill chosen 20% of the time
Composition Failure	Developer cannot determine if Skill A chains into Skill B	No input/output contracts, no composability metadata	Manual trial-and-error; broken pipelines; skills used in isolation when they should chain
Activation Failure	Agent context polluted by irrelevant skills; hallucinated capabilities	All skills loaded simultaneously; no activation profiles	Agent quality degrades as library grows; developer distrusts agent suggestions

Table 1: The Three Systemic Failures of Unmanaged Skill Libraries

2. Competitive Analysis: Why Existing Solutions Fall Short

Several categories of tools address parts of the skill management problem, but none address all three failure modes (discovery, composition, activation) in a unified system. This section analyzes the closest competitors across four categories: visual workflow builders, DAG orchestrators, skill registries, and agent frameworks. For each, we identify what they do well, where they fall short, and why SkillFlow's approach is fundamentally different.

2.1 Visual Workflow Builders

Node-RED

Node-RED is the pioneer of flow-based programming for IoT and automation. It organizes nodes in a left-side palette grouped by category and color-codes them on the canvas (coral for input, yellow for function, blue for output, green for routing). This two-axis system (palette

position + canvas color) is excellent for discoverability within Node-RED's domain. However, Node-RED fails on three critical fronts for skill management. First, it has no concept of composability classes: every node is treated equally, with no distinction between "noun" nodes that produce output and "adjective" nodes that modify other nodes. Second, its color system is fixed and shallow (4-5 colors with no secondary encoding for saturation or lightness), providing no mechanism to encode the noun/verb/adjective distinction that is central to skill composition. Third, and most fundamentally, Node-RED is a runtime execution engine, not a management layer. It assumes all nodes are always available and always compatible, which is precisely the assumption that breaks at skill-library scale.

n8n

n8n extends the visual workflow model with a five-part node taxonomy (triggers, actions, connectors, AI nodes, storage) and a community of 7,000+ workflow templates. Its trigger/action/connector separation is the closest existing analog to our noun/verb classification. However, n8n's taxonomy is rigid: every node must fit into one of five buckets, with no mechanism for adjective/modifier nodes that enhance other nodes without producing independent output. The adjective pattern (skills like "add-citations," "add-reasoning," "add-error-recovery") is the most common and most valuable composition pattern in AI skill systems, yet n8n has no way to represent it. Additionally, n8n is a SaaS product with a visual-only authoring experience; it cannot be run as a local CLI tool or integrated into an agent's skill-loading process.

ComfyUI

ComfyUI is the most directly relevant system because it chains AI capabilities into visual pipelines (load model, encode prompt, sample, decode, save). Its node categories (loaders, samplers, latent, conditioning) map closely to AI skill domains, and its JSON serialization enables sharing and version control. ComfyUI's fundamental limitation is that it is domain-locked to image generation. Its node types, data flow semantics, and validation rules are all designed for the specific pipeline structure of diffusion models. There is no mechanism to generalize its approach to the broader skill ecosystem, where skills operate on text, code, documents, web pages, and deployment targets. ComfyUI also lacks a classification system: nodes are organized by functional category but not by composability class, and there is no equivalent to the noun/verb/adjective distinction that enables predictive pipeline composition.

Dify / Flowise

Dify and Flowise are AI workflow builders with drag-and-drop visual editors. Dify categorizes building blocks into LLM nodes, tool nodes, knowledge retrieval, iteration, and conditional routing. Flowise provides modular building blocks from LangChain. Both are cloud-first,

graphical-only tools designed for building single-purpose automation workflows rather than managing a library of reusable skills. Neither provides a taxonomy, a color-coding system, an activation manager, or a CLI for scanning and classifying existing skills. They are workflow builders, not skill operating systems.

2.2 DAG Orchestrators

Apache Airflow / Dagster / Prefect

These systems excel at defining, scheduling, and monitoring directed acyclic graphs of tasks. Airflow uses Python code with bitshift operators for dependency declaration; Dagster introduces Software-Defined Assets with type-checked data flow; Prefect emphasizes runtime dynamism and failure handling. All three are production-grade infrastructure for running pipelines at scale. However, they are fundamentally execution engines, not management layers. They assume you already know which tasks to compose and how they connect. They provide no mechanism for discovering skills, no visual cheat sheet for browsing your library, no color-coding system for identifying skill types at a glance, and no activation manager for loading context-appropriate subsets. Using Airflow to manage skills would be like using a freight train schedule to organize your pantry: the infrastructure is impressive, but the problem is wrong.

2.3 Skill Registries

SkillNet (Zhejiang University)

SkillNet is the most academically rigorous skill infrastructure, treating AI skills as first-class packages organized in a structured network with similarity, composition, and dependency relations across 200,000+ skills. It models skills as composable assets with typed relations, which is conceptually close to our approach. However, SkillNet operates at web scale (200K+ skills) and is designed for skill discovery across a global network, not for managing a local library of 50-200 skills. It provides no visual cheat sheet, no color-coding taxonomy, no pipeline DSL for local composition, and no activation manager. Its interface is API-based and research-oriented, not CLI-based and developer-friendly. SkillNet solves the "find skills on the internet" problem; we solve the "find skills on your machine" problem.

VoltAgent / SkillsLLM / Skify

These are emerging marketplaces and registries for AI agent skills. VoltAgent curates 1,000+ skills; SkillsLLM offers 1,600+ security-vetted skills; Skify positions itself as "npm for agent workflows." All three address skill distribution and discovery, but none provides a local management layer. They are registries, not operating systems. Installing skills from a registry without a local taxonomy is like buying books without a bookshelf: you accumulate without

organizing, and the collection becomes less useful as it grows.

2.4 Agent Frameworks

LangGraph / CrewAI / AutoGen

LangGraph provides graph-based agent orchestration with state machines. CrewAI organizes multi-agent teams around roles. AutoGen enables multi-agent conversations for iterative problem-solving. These frameworks define how agents interact, but they do not manage the skills that agents use. A CrewAI agent that has been assigned the "frontend developer" role still needs to discover, select, and compose the right skills from the library. The framework provides the orchestration pattern (who talks to whom) but not the skill management layer (which skills are available, how they compose, and which to activate). SkillFlow is complementary to these frameworks: it provides the skill operating system that agent frameworks lack.

2.5 Competitive Summary

Capability	Node-RED	n8n	ComfyUI	SkillNet	SkillFlow
Local skill scanning	No	No	No	No	Yes
7-hue color taxonomy	4 colors	5 categories	No taxonomy	No	Yes (7 hues + secondary)
Noun/Verb/Adjective classes	No	No	No	Partial	Yes (core model)
Input/output contracts	Typed ports	Partial	Typed	Relations	Yes (declarative)
Pipeline DSL (YAML)	JSON	JSON	JSON	No	Yes (YAML + validation)
Activation profiles	No	No	No	No	Yes (context-based)
Visual cheat sheet	Palette	Palette	No	No	Yes (A3 PDF)
CLI-first, local-first	No (server)	No (SaaS)	No (server)	No (API)	Yes
Adjective/modifier support	No	No	No	No	Yes (first-class)
Pipeline validation	Runtime	Runtime	Runtime	No	Pre-flight (static)

Table 2: Competitive Capability Matrix

3. How SkillFlow Does It Correctly

SkillFlow solves all three failure modes through four interlocking innovations that no existing system combines: a composability-aware taxonomy, an opponent-process color system, a pre-flight pipeline validator, and context-based activation profiles. Each innovation addresses a specific failure mode while reinforcing the others, creating a system where discovery, composition, and activation are not three separate features but three views of the same underlying model.

3.1 Composability-Aware Taxonomy (Solves Discovery + Composition)

The Noun/Verb/Adjective framework is the conceptual foundation of SkillFlow. Unlike n8n's five-part taxonomy (which has no modifier class) or Node-RED's color-only distinction (which has no composability semantics), the NVA framework directly encodes how skills compose. Noun skills are standalone producers (Factory pattern): they generate artifacts from minimal input. Verb skills are transformers (Strategy pattern): they take an artifact and produce a modified artifact. Adjective skills are modifiers (Decorator pattern): they wrap other skills and enhance their behavior without replacing them. This is not just a labeling system; it is a predictive model. If you know a skill is a noun, you know it can start a pipeline. If you know it is a verb, you know it needs an upstream input. If you know it is an adjective, you know it can be attached to any compatible skill in any pipeline. No existing system provides this level of composability reasoning.

> **Why Competitors Miss This**
> Node-RED and n8n were designed for data flow, not skill composition. In data flow, every node is a transformer (verb); there are no "adjective" nodes that modify other nodes' behavior. The adjective pattern is unique to AI skill systems, where skills like "add-reasoning" or "add-retry" wrap other skills rather than producing independent output. SkillFlow is the first system to make adjective skills first-class citizens in the taxonomy, with their own color (teal), their own icon (star), and their own composition rules (attach to any noun or verb).

3.2 Opponent-Process Color System (Solves Discovery)

SkillFlow uses a 7-hue opponent-process color system based on Hering's opponent-process theory and validated by ColorBrewer's research on categorical palettes. The system encodes domain category (7 hues), composability class (3 saturation levels), and operational status (2 lightness levels) in a multi-channel visual encoding. This is fundamentally more expressive than Node-RED's 4-5 flat colors or KNIME's category-only encoding. The opponent-pair structure (blue/orange for input/output, green/red for transform/control) provides semantic meaning: when you need a skill that feeds data, you look for blue; when you need one that produces a deliverable, you look for orange. Colors that are opposite on the wheel

correspond to skills that are opposite in the pipeline, making the cheat sheet a functional map, not just a decorative reference. No competitor uses opponent-process semantics or multi-channel encoding.

3.3 Pre-Flight Pipeline Validator (Solves Composition)

Existing systems validate pipelines at runtime (when you run the workflow and it fails). SkillFlow validates pipelines at design time, before any skill is invoked. The validator checks three things: (1) input/output contract compatibility between adjacent skills, ensuring that Skill A's output matches what Skill B expects as input; (2) acyclicity, ensuring no circular dependencies; and (3) adjective compatibility, ensuring that modifier skills are attached to skills they can actually enhance. This pre-flight approach catches composition errors before they waste developer time, and it enables the pipeline DSL to serve as executable documentation that can be version-controlled, shared, and reviewed. Airflow and Dagster validate at the task level, but they do not validate skill-level composability because they have no model of what skills are or how they compose.

3.4 Context-Based Activation Profiles (Solves Activation)

SkillFlow introduces activation profiles that load a curated subset of skills based on the current project context. A "frontend" profile loads all HIGH and MEDIUM front-end skills and disables the rest. A "research" profile loads research and data-enrichment skills. Profiles can be composed ("frontend + research") and switched with a single CLI command. No existing skill management system provides this capability. Node-RED loads all installed nodes. n8n loads all connectors. ComfyUI loads all custom nodes. SkillNet has no concept of a local activation state. The activation manager directly addresses the context-pollution problem: instead of loading 100+ skills into the agent's context window (which degrades agent quality), you load 20-30 relevant skills (which focuses the agent on the right capabilities).

3.5 Differentiation Summary

What Others Do	Why It Fails	What SkillFlow Does
Flat alphabetical skill list	Exceeds Miller's 7+/-2; no visual cues; scan-dependent	7-hue color taxonomy with composability-aware grouping
Category-only color coding (Node-RED, KNIME)	Encodes domain but not role; no composability semantics	Multi-channel: hue=domain, saturation=class, lightness=status
5-part node taxonomy (n8n)	No modifier/adjective class; rigid categories; no NVA model	Noun/Verb/Adjective with Decorator-pattern semantics

What Others Do	Why It Fails	What SkillFlow Does
Runtime pipeline validation (Airflow, n8n)	Errors discovered during execution; wasted compute and time	Pre-flight static validation of contracts, acyclicity, compatibility
All skills always loaded	Context pollution; agent quality degrades with library size	Activation profiles load context-appropriate subsets
Cloud/SaaS-only (Dify, Flowise, n8n)	No local CLI; no offline capability; no skill scanning	CLI-first, local-first; scans your actual skill directory
Global registry only (SkillNet, SkillsLLM)	Solves "find on internet" not "find on your machine"	Local manifest from scanning your actual installed skills

Table 3: Differentiation Matrix: How SkillFlow Differs from Existing Solutions

4. User Personas

Attribute	Alex (Frontend Dev)	Sam (Full-Stack)	Jordan (Team Lead)
Role	Front-end developer	Full-stack engineer	Engineering manager
Skills installed	80-120	100-200	Monitors team library
Primary pain	Cannot find animation skills fast enough	Cannot tell if skills compose	No way to standardize team workflows
Discovery style	Visual; wants a cheat sheet	Search-first; wants CLI	Documentation-first; wants templates
Pipeline usage	Builds same 3-4 pipelines daily	Experiments with new combinations	Defines standard pipelines for team
Activation need	Switch between "frontend" and "design" profiles	Switch between "backend" and "devops" profiles	Enforce team profile standards

Table 4: User Personas

5. User Experience Stories

The following user stories are organized by the three failure modes they address: discovery, composition, and activation. Each story includes acceptance criteria that define the minimum viable behavior for the feature. Priority levels (P0/P1/P2/P3) indicate implementation order, with P0 being required for the MVP and P3 being future enhancements.

5.1 Discovery Stories

US-D1: Scan and classify skills [P0]

As a developer with an existing skill library, I want to run a single command that scans my skills directory and produces a classified manifest, so that I can see at a glance what skills I have, what they do, and how they are categorized.

AC1: skill-scan reads all SKILL.md files in the configured directory
AC2: Each skill is assigned a domain (7 categories), class (noun/verb/adjective), and color
AC3: Output is a manifest.json with all classification metadata
AC4: Heuristic classifier achieves >80% accuracy vs. manual classification
AC5: Manual overrides via YAML front matter take precedence over heuristics

US-D2: Search skills by keyword, category, or trait [P0]

As a developer in the middle of a task, I want to search for skills by what they do, not just by name, so that I can find the right skill in under 5 seconds without scanning the entire list.

AC1: skill-search accepts keywords, domain names, class names, and trait tags
AC2: Results are ranked by relevance and front-end utility rating
AC3: Search completes in under 1 second for libraries up to 500 skills
AC4: Color-coded output shows domain, class, and FE relevance at a glance

US-D3: Generate visual cheat sheet [P1]

As a developer who wants a quick reference, I want to generate a single-page visual cheat sheet with all my skills grouped and color-coded, so that I can find any skill by looking at the sheet instead of scanning a text list.

AC1: skill-cheatsheet generates an A3 PDF with all skills from the manifest
AC2: Skills are grouped by domain (7 color columns) and sub-ordered by class
AC3: Each skill card shows name, one-line description, color swatch, and class icon
AC4: Adjective skills are highlighted in a floating sidebar with teal borders
AC5: Pipeline templates are rendered as small flow diagrams at the bottom
AC6: Legend explains the color and icon system

US-D4: Identify skill gaps in my library [P2]

As a developer building a pipeline, I want to see which skill domains are under-represented in my library, so that I can make informed decisions about which skills to acquire next.

AC1: skill-scan --gaps reports domains with fewer than 3 skills
AC2: Suggests pipeline stages that cannot be completed due to missing skills
AC3: Cross-references pipeline templates against available skills

5.2 Composition Stories

US-C1: Define a pipeline in YAML [P0]

As a developer who has identified a repeatable workflow, I want to define a skill pipeline in a YAML file with clear skill references and data flow, so that I can version-control, share, and reuse my workflows instead of re-composing them manually.

AC1: Pipeline YAML supports chain, fan-out/fan-in, route, orchestrate, and evaluate-loop patterns
AC2: Each skill reference is validated against the manifest (must exist, must be installed)
AC3: Input/output contracts between adjacent skills are checked for compatibility
AC4: Adjective skills can be declared as modifiers on specific pipeline stages
AC5: Validation errors are reported with clear messages and suggested fixes

US-C2: Validate a pipeline before execution [P0]

As a developer about to run a multi-skill pipeline, I want to validate my pipeline definition against the skill manifest before executing it, so that I can catch composition errors early and avoid wasting time on broken pipelines.

AC1: skill-pipeline validate checks contract compatibility between all adjacent skills
AC2: Detects circular dependencies and reports them with the specific cycle
AC3: Verifies that all adjective skills are compatible with their target skills
AC4: Reports warnings for potentially fragile compositions (e.g., multiple adjectives on one stage)
AC5: Returns exit code 0 for valid pipelines, 1 for errors, 2 for warnings only

US-C3: Browse pre-defined pipeline templates [P1]

As a developer new to the skill ecosystem, I want to browse a library of pre-defined pipeline templates for common workflows, so that I can get started quickly without having to design my own pipeline from scratch.

AC1: Built-in templates include: frontend-build, research-report, code-review, design-to-deploy
AC2: skill-pipeline list-templates shows available templates with descriptions
AC3: skill-pipeline init --template creates a local YAML from the template
AC4: Templates can be customized after initialization

US-C4: Discover adjective skills that enhance my current pipeline [P2]

As a developer who has built a working pipeline, I want to see which adjective skills can be added to enhance any stage of my pipeline, so that I can systematically improve pipeline quality without redesigning it.

AC1: skill-pipeline suggest-adjectives analyzes the current pipeline and recommends modifiers
AC2: Recommendations are based on the domain and class of each pipeline stage
AC3: Each recommendation includes the specific enhancement it would provide
AC4: Developer can add adjectives to the pipeline YAML with a single command

5.3 Activation Stories

US-A1: Switch activation profiles [P0]

As a developer switching between project types, I want to switch my active skill set with a single command that loads a context-appropriate profile, so that my agent only sees relevant skills and does not waste context on irrelevant ones.

AC1: skill-activate loads skills matching the profile into the agent context
AC2: Built-in profiles include: frontend, backend, devops, research, design
AC3: Profiles can be composed: skill-activate frontend+research
AC4: skill-activate list shows available profiles and their skill counts
AC5: Current active profile is persisted and restored on next session

US-A2: Create custom activation profiles [P1]

As a developer with specialized workflow needs, I want to define custom activation profiles that select skills by domain, class, and relevance, so that I can create profiles tailored to my specific project types and team standards.

AC1: Custom profiles defined in config/profiles.yaml
AC2: Profile rules support: domain filters, class filters, relevance thresholds, explicit includes/excludes
AC3: skill-activate --dry-run shows which skills would be loaded without loading them
AC4: Profile validation warns if critical skills are excluded

US-A3: Auto-suggest profile based on project context [P2]

As a developer starting a new project, I want SkillFlow to suggest an activation profile based on the files and technologies in my project, so that I do not have to manually select a profile every time I switch projects.

AC1: skill-activate --auto scans the current project directory for technology indicators
AC2: Detects: package.json (Node), requirements.txt (Python), next.config (Next.js), etc.
AC3: Suggests the most appropriate profile based on detected technologies
AC4: Developer can accept, modify, or reject the suggestion

6. Functional Requirements

ID	Requirement	Priority	Phase
FR-01	Scan skills directory and parse SKILL.md files	P0	1
FR-02	Classify each skill by domain (7 categories)	P0	1
FR-03	Classify each skill by composability (noun/verb/adjective)	P0	1
FR-04	Generate manifest.json with all classification metadata	P0	1
FR-05	Support YAML front matter overrides in SKILL.md	P0	1
FR-06	Search skills by keyword, domain, class, and trait	P0	1
FR-07	Color-code output by 7-hue opponent process system	P1	1
FR-08	Generate A3 visual cheat sheet PDF	P1	2
FR-09	Generate HTML cheat sheet (editable source)	P1	2
FR-10	Define pipeline in YAML with 5 composition patterns	P0	3
FR-11	Validate pipeline: contract compatibility	P0	3
FR-12	Validate pipeline: acyclicity check	P0	3
FR-13	Validate pipeline: adjective compatibility	P1	3
FR-14	Built-in pipeline templates (4 minimum)	P1	3
FR-15	Switch activation profiles via CLI	P0	4
FR-16	Compose profiles (e.g., frontend+research)	P0	4

ID	Requirement	Priority	Phase
FR-17	Custom profile definition in YAML	P1	4
FR-18	Persist and restore active profile across sessions	P1	4
FR-19	Suggest adjective skills for pipeline enhancement	P2	3
FR-20	Auto-detect project type and suggest profile	P2	4
FR-21	React Flow visual pipeline editor (web)	P3	5
FR-22	Skill gap analysis across pipeline templates	P2	2

Table 5: Functional Requirements

7. Non-Functional Requirements

ID	Requirement	Target	Rationale
NFR-01	Scan performance	100 skills in under 5 seconds	Developer runs scan frequently; must not block workflow
NFR-02	Search performance	Results in under 1 second	Must feel instant; Hick's Law applies to tool selection
NFR-03	Classification accuracy	Heuristic >80% vs. manual	Reduces manual override burden; developer trust requires accuracy
NFR-04	Color accessibility	All 7 hues distinguishable under CVD	Opponent-process pairs (blue/orange) maximize CVD discriminability
NFR-05	CLI-first architecture	All core features accessible via CLI	Developers live in the terminal; GUI is Phase 5 bonus
NFR-06	Local-first data	All data stored locally in ~/.skillflow/	No cloud dependency; works offline; respects privacy
NFR-07	Pipeline validation speed	Validate 20-skill pipeline in under 2 seconds	Pre-flight checks must not add meaningful delay
NFR-08	Manifest portability	manifest.json is self-contained and shareable	Teams can share manifests; CI can validate pipelines
NFR-09	Backward compatibility	SKILL.md files work without front matter	Existing 1000+ skills require zero modification to be scanned
NFR-10	Extensibility	Custom domains, colors, and traits configurable via YAML	Different teams may need different taxonomy axes

Table 6: Non-Functional Requirements

8. Technical Architecture

SkillFlow is a CLI-first, local-first Python application that scans, classifies, validates, and visualizes AI agent skills. The architecture follows a pipeline-of-pipelines pattern: the tool itself is a pipeline (scan, classify, validate, render) that manages other pipelines (skill compositions defined in YAML). All data is stored locally in ~/.skillflow/, and no cloud services are required for core functionality.

8.1 File Structure

```
~/.skillflow/
manifest.json # Generated skill manifest (single source of truth)
config/
taxonomy.yaml # Domain definitions, color mapping, class rules
profiles.yaml # Activation profile definitions
pipelines.yaml # Pre-defined pipeline templates
overrides.yaml # Manual classification overrides
output/
cheatsheet.html # Generated HTML cheat sheet
cheatsheet.pdf # Generated PDF cheat sheet
pipelines/ # Generated pipeline diagrams (Mermaid/HTML)
cache/
skill_hashes.json # Content hashes for incremental scanning
```

8.2 CLI Commands

Command	Description	Phase
skill-scan [--dir PATH]	Scan and classify all skills; generate/update manifest	1
skill-search QUERY [--domain D] [--class C] [--trait T]	Search skills by keyword, domain, class, or trait	1
skill-cheatsheet [--format pdf|html]	Generate visual cheat sheet from manifest	2
skill-pipeline init [--template NAME]	Initialize a pipeline YAML from template	3
skill-pipeline validate PIPELINE.yaml	Validate pipeline against manifest	3
skill-pipeline run PIPELINE.yaml	Execute a validated pipeline (Phase 3+)	3
skill-pipeline suggest-adjectives PIPELINE.yaml	Recommend adjective skills for pipeline	3
skill-activate PROFILE	Activate a skill profile for current context	4
skill-activate list	List available profiles and skill counts	4
skill-activate --auto	Auto-detect project type and suggest profile	4

Table 7: CLI Command Reference

8.3 Skill Schema

Each skill in the manifest is represented by a JSON object with the following fields. Fields marked "auto" are inferred by the heuristic classifier; fields marked "manual" are set via YAML front matter in the SKILL.md file. Manual values always override auto-inferred values.

Field	Type	Source	Description
name	string	auto (dirname)	Skill name from directory name
domain	enum (7 values)	auto + manual override	Primary domain category (maps to color hue)
class	enum (noun/verb/adjective)	auto + manual override	Composability class (maps to saturation)
description	string	auto (first paragraph)	One-line description from SKILL.md
input_contract	string	manual	What the skill consumes
output_contract	string	manual	What the skill produces
pipeline_tags	list[string]	auto + manual	Compatible composition patterns
traits	list[string]	manual	Faceted filter tags (e.g., requires-api, cacheable)
relevance	enum (high/med/low)	manual	Domain-specific utility rating
color	hex	computed	Derived from domain + class + status
status	enum (active/dormant)	computed	Based on current activation profile

Table 8: Skill Manifest Schema

8.4 Pipeline DSL

Pipelines are defined in YAML, inspired by the CNCF Serverless Workflow Specification v1.0. The DSL supports five composition patterns (chain, fan-out/fan-in, route, orchestrate, evaluate-loop), typed skill references validated against the manifest, adjective modifier declarations, and data flow annotations. The validator checks contract compatibility, acyclicity, and adjective compatibility before execution. Example pipeline:

```
name: frontend-build-pipeline
description: "Complete front-end build from design to deployment"
pattern: chain
skills:
- id: design
skill: ui-ux-pro-max
input: { brief: "$.user.brief" }
```

```
output: { tokens: "$.design.tokens" }
- id: animate
skill: auto-animate
input: { components: "$.design.tokens" }
modifiers: [gsap-scrolltrigger]
- id: test
skill: webapp-testing
modifiers: [testing-patterns]
- id: deploy
skill: vercel-deploy
modifiers: [ship-workflow]
adjectives:
- skill: vercel-react-best-practices
apply_to: [design, animate]
- skill: damage-control
apply_to: [deploy]
```

9. Implementation Phases

Phase	Name	Deliverables	Timeline	Dependencies
1	Scanner	skill-scan + skill-search CLI + manifest.json	2 weeks	Python 3.10+, PyYAML
2	Cheat Sheet	skill-cheatsheet CLI + A3 PDF + HTML	2 weeks	Phase 1, Playwright, WeasyPrint
3	Pipeline Composer	skill-pipeline CLI + DSL + validator + templates	3 weeks	Phase 1, YAML parser, jsonschema
4	Activator	skill-activate CLI + profiles + auto-detect	2 weeks	Phase 1, agent config access
5	Visual Editor	React Flow web UI for pipeline composition	4-6 weeks	Phase 2+3, React, xyflow

Table 9: Implementation Phases

The phases are designed to deliver independent value at each stage. Phase 1 (Scanner) alone solves the discovery problem by providing a classified, searchable manifest. Phase 2 (Cheat Sheet) adds the visual reference that makes discovery instant. Phase 3 (Pipeline Composer) solves the composition problem with a validated DSL. Phase 4 (Activator) solves the activation problem with context-based profiles. Phase 5 (Visual Editor) is the aspirational capstone that

brings all four capabilities into a drag-and-drop web interface, but it is not required for the core value proposition. A developer working alone can be fully productive with Phases 1-4 alone.

10. Success Metrics

The following metrics measure whether SkillFlow is solving the three failure modes. Each metric has a baseline (current state without SkillFlow) and a target (state with SkillFlow). Metrics are measured through a combination of CLI telemetry (opt-in), user surveys, and timed task completion studies.

Failure Mode	Metric	Baseline	Target	Measurement
Discovery	Time to find the right skill	15-30 seconds	< 5 seconds	Timed task: "find the skill that does X"
Discovery	Wrong skill selection rate	~20%	< 5%	Survey: "did the skill you chose do what you expected?"
Composition	Pipeline construction time	10-20 minutes	< 3 minutes	Timed task: "build pipeline X from scratch"
Composition	Pipeline failure rate (first run)	~40%	< 10%	Telemetry: validate vs. runtime error ratio
Activation	Agent context pollution score	High (100+ skills)	Low (20-30 skills)	Profile skill count vs. total library count
Activation	Agent suggestion accuracy	Moderate	High	Survey: "does the agent suggest the right skill?"

Table 10: Success Metrics

Summary
SkillFlow is the first skill management system designed for the AI agent era. It solves discovery through a 7-hue opponent-process color taxonomy and visual cheat sheets, composition through a Noun/Verb/Adjective framework and pre-flight pipeline validation, and activation through context-based profiles that load only relevant skills. No existing system combines all three capabilities, and no existing system treats adjective/modifier skills as first-class citizens. SkillFlow is CLI-first, local-first, and designed to work with the SKILL.md format that the community has already adopted, requiring zero modification to existing skills.